SEMESTER PROJECT REPORT

PINDI KI KHOJ

Local Business & Service Finder for Rawalpindi

Submitted By

Muhammad Ali Farrukh 241896

Hafiz Danial Ahmed Khan 241881

Muhammad Farhan Adil 241860

Visual Programming — 4th Semester

Department of Computer Science

20 May 2026

1. Introduction

Pindi Ki Khoj (literally “Discovery of Rawalpindi”) is a web-based platform that connects local service providers — mechanics, tutors, electricians, restaurants, and other professionals — directly with customers in Rawalpindi. The platform addresses a common urban problem: residents often struggle to find trustworthy, nearby service providers quickly, while skilled professionals lack an affordable digital channel to reach customers.

The application was developed as a 4th-semester Visual Programming project using ASP.NET Core Blazor. It goes beyond a simple business directory by incorporating real-time negotiation, live GPS tracking, and direct in-app messaging, making it a full end-to-end service marketplace.

1.1 Problem Statement

In Rawalpindi, most local businesses still rely on word-of-mouth or physical signboards for visibility. Customers have no convenient way to search for available service providers near them, compare rates, or communicate before committing to a booking. This disconnect leads to wasted time for both parties.

1.2 Proposed Solution

Pindi Ki Khoj provides a three-role platform (Admin, Business Owner, Customer) where service providers register their businesses, customers discover and contact them on a live map, negotiate prices through in-app chat, and track the provider’s arrival via GPS — all in one seamless experience.

2. Project Objectives

- Enable customers to discover nearby, approved businesses filtered by service category and distance using GPS coordinates.
- Provide real-time price negotiation between customers and service providers through an integrated chat system.
- Implement live GPS tracking so customers can monitor a service provider’s location during active service requests.
- Build a role-based access control system supporting three distinct user types: Admin, Business Owner, and Customer.
- Allow business owners to self-register and manage their profile, pending admin approval before going live.
- Let customers submit reviews and ratings after service completion to build provider credibility.
- Give administrators a centralised dashboard to manage users, businesses, and service requests.

3. Technology Stack

The application uses a modern, cohesive .NET ecosystem to deliver both the frontend and backend from a single codebase.

Layer	Technology	Purpose
Frontend / UI	Blazor Server (.NET 8)	Interactive component-based web UI rendered server-side
Styling	Bootstrap 5, Custom CSS	Responsive layout and visual design
Maps	Leaflet.js	Interactive maps for business discovery and GPS tracking
Real-Time Comms	SignalR	WebSocket-based chat and live GPS location streaming
Backend	ASP.NET Core 8	HTTP pipeline, dependency injection, middleware
ORM / Data	Entity Framework Core	Database modelling, migrations, and LINQ queries
Database	SQLite / SQL Server	Switchable via configuration flag; defaults to SQL Server LocalDB
Authentication	ASP.NET Core Identity	User registration, login, password hashing, roles

4. System Architecture

Pindi Ki Khoj follows a layered server-side architecture. All rendering occurs on the server; the browser maintains a persistent SignalR connection for component interactivity, chat, and GPS updates.

4.1 Application Layers

Layer	Components
Presentation (Razor Components)	Pages: Home, Discover, CreateRequest, MyRequests, Dashboard (User, Owner, Admin), Login, Register
Shared UI Components	LiveMap.razor (Leaflet map), RequestDetail.razor (shared request view), Layout files per role
SignalR Hubs	ChatHub.cs — real-time messaging; TrackingHub.cs — GPS location broadcast
Service Layer	BusinessService, RequestService, AdminService, GeoHelper
Data Access	ApplicationDbContext (EF Core), DbInitializer (seed data), Migrations
Domain Models	Business, ServiceRequest, ServiceCategory, ChatMessage, LocationPing, Review, ApplicationUser

4.2 Role-Based Access

The application enforces three distinct roles through ASP.NET Core Identity and route-level authorization:

Role	Key Permissions
Admin	Approve / reject business registrations, manage all users (activate / deactivate), view full system dashboard and all service requests
Business Owner	Register a business, view incoming service requests, accept / negotiate / complete requests, share live GPS location
Customer (User)	Browse and discover nearby businesses on a live map, submit service requests, negotiate price via chat, track provider location, leave reviews

5. Database Design

5.1 Entity Relationship Overview

The database is built with Entity Framework Core Code-First migrations. The main entities and their relationships are:

Entity	Key Fields	Relationships
ApplicationUser	FullName, LastLatitude, LastLongitude, IsActive, CreatedAt	One-to-One with Business (Owner); One-to-Many with ServiceRequest
ServiceCategory	Name, Description, Icon (Bootstrap Icons class)	One-to-Many with Business
Business	Name, Address, Phone, HourlyRate, Latitude, Longitude, IsApproved, IsOnline, Rating	Belongs to ApplicationUser and ServiceCategory; has many ServiceRequests and Reviews
ServiceRequest	Title, Description, Status (enum), VisitType (enum), AgreedPrice, UserLatitude/Longitude	Belongs to User and Business; has many ChatMessages, LocationPings; one Review
ChatMessage	Content, IsOffer, OfferAmount, SentAt	Belongs to ServiceRequest and Sender (ApplicationUser)
LocationPing	Latitude, Longitude, Timestamp	Belongs to ServiceRequest and User
Review	Rating (1-5), Comment, CreatedAt	Belongs to Business, ServiceRequest, and User (one review per completed request)

5.2 Service Request Lifecycle

ServiceRequest.Status is modelled as an enum with seven states, forming a clear state machine:

Status	Meaning
Pending	Request submitted by customer, awaiting owner response
Negotiating	First chat message sent; price discussion in progress
DealAgreed	Both parties confirmed price and visit type (owner goes to user or vice versa)
InProgress	Service underway; GPS tracking active
Completed	Service finished; customer can now leave a review
Cancelled	Request cancelled by either party

6. Key Features & Implementation

6.1 Business Discovery with Live Map

The Discover page uses Leaflet.js (loaded via JavaScript interop) to render an interactive map centred on the user's GPS position. The BusinessService.SearchNearbyAsync() method fetches approved, online businesses and filters them using the Haversine formula implemented in GeoHelper.DistanceKm(), sorting results by distance. Businesses are rendered as clickable map markers alongside a sidebar list.

6.2 Real-Time Chat & Price Negotiation

Once a customer selects a business and submits a service request, a dedicated chat thread is opened. ChatHub.cs (a SignalR hub marked [Authorize]) handles two operations: JoinRequest — adds the connection to a SignalR group named request-{id} — and SendMessage — persists the ChatMessage via RequestService and broadcasts it to all group members. Offer messages carry an optional OfferAmount, enabling structured price proposals within the chat stream.

6.3 Live GPS Tracking

TrackingHub.cs implements location sharing during in-progress requests. The browser's Geolocation API (called via JS interop in maps.js) supplies coordinates, which are sent to UpdateLocation on the hub. Each ping is persisted as a LocationPing record and the user's last known position (ApplicationUser.LastLatitude / LastLongitude) is updated. All members of the tracking group receive a LocationUpdated event and the Leaflet map updates the marker in real time.

6.4 Admin Panel

AdminService exposes a DashboardStats aggregate (total users, businesses, pending approvals, active and completed requests) displayed on the Admin Dashboard. Admins can approve or reject pending business registrations and activate or deactivate user accounts, giving full platform governance without touching the database directly.

6.5 Reviews & Ratings

After a request is marked Completed, the customer can submit a 1-5 star rating and a written comment. RequestService.AddReviewAsync() saves the Review and recalculates Business.Rating and Business.ReviewCount as a live average, ensuring the map and listing always show up-to-date scores. One review is enforced per completed request through a unique foreign key constraint.

7. Project Structure

The solution follows the default Blazor Server project layout with clear separation of concerns:

Folder / File	Contents
Components/Pages/	Role-segregated pages: Account (Login, Register, Logout), Admin, Owner, User, Shared
Components/Layout/	Four layout files: MainLayout, AdminLayout, OwnerLayout, UserLayout — each with its own navigation
Components/Shared/	LiveMap.razor (reusable Leaflet map), RedirectToLogin.razor
Models/	Six domain model classes: Business, ServiceRequest, ServiceCategory, ChatMessage, LocationPing, Review
Data/	ApplicationDbContext, ApplicationUser, DbInitializer (seeder), Migrations, DatabaseExtensions
Services/	BusinessService, RequestService, AdminService, GeoHelper, IdentityRevalidatingAuthenticationStateProvider
Hubs/	ChatHub.cs, TrackingHub.cs (both SignalR hubs, both [Authorize])
Constants/	AppRoles.cs — defines Admin, User, BusinessOwner role name constants
wwwroot/	app.css, favicon, logo, Leaflet JS integration (app.js, maps.js), Bootstrap 5 distribution

8. Challenges & Solutions

Challenge	Solution Adopted
Geo-distance filtering in database	Fetched all approved/online businesses into memory, then applied Haversine formula (GeoHelper) server-side to avoid unsupported spatial SQL functions
Real-time updates in Blazor Server	Used SignalR groups (one per request ID) so only the relevant users receive chat and GPS events, avoiding unnecessary broadcasts
Database portability	Introduced a runtime flag (--sqlite / ASPNETCORE_ENVIRONMENT_SQLITE=1) that swaps the EF Core provider between SQL Server LocalDB and SQLite without code changes
Authorization in SignalR hubs	Applied [Authorize] attribute on hub classes and added a CanAccessAsync check on every hub method to ensure users can only join groups for their own requests
Review integrity	Enforced one-review-per-request at the database level via a one-to-one foreign key between Review and ServiceRequest, preventing duplicate reviews

9. Setup & Running the Application

Prerequisites: .NET 8 SDK, Visual Studio 2022 (or VS Code with C# Dev Kit).

1. Clone the repository and open LocalBusinessFinder.sln in Visual Studio.
2. Press F5 to build and run. EF Core migrations execute automatically on first launch, creating the database and seeding default roles and categories.
3. To use SQLite instead of SQL Server LocalDB, run: dotnet run -- --sqlite
4. Navigate to https://localhost:{port} in the browser. Register as a new user, or use the seeded Admin account to approve businesses.

10. Conclusion

Pindi Ki Khoj successfully demonstrates how a modern ASP.NET Core Blazor application can combine multiple advanced features — real-time communication, geolocation, role-based access control, and a relational database — into a cohesive, production-grade product.

The project provided hands-on experience with SignalR WebSockets, Entity Framework Core migrations, Leaflet.js map integration, and ASP.NET Core Identity. These technologies reflect real-world industry practices, making the project a valuable preparation for professional software development.

Future enhancements could include push notifications, a mobile application, payment gateway integration, and advanced analytics for business owners.

Appendix: Technology Versions

Package / Framework	Version
ASP.NET Core / .NET SDK	8.0
Blazor Server	8.0
Microsoft.AspNetCore.SignalR	8.0 (bundled with ASP.NET Core)
Entity Framework Core	9.x
Microsoft.AspNetCore.Identity	8.0 (bundled)
Bootstrap	5.x
Leaflet.js	Latest stable (CDN)
SQLite (Microsoft.Data.Sqlite)	9.x